

Changelog

All notable changes to the Claude multi-account toolkit.

v1.7.6 — 2026-06-28 — Always-non-interactive execution, alias-file integrity, macOS/bash-3.2 portability, 4-host rollout

Fixed

- **Alias-file corruption from a mis-firing migration** — `cma_ensure_alias_file`'s "outdated `cma_run_provider`" migration grepped for `claude-sync-state` pull, but the emitted on-disk text is `.../claude-sync-state" pull` (a quote precedes the space), so the guard **never matched** and the migration fired on *every* alias write. Its `awk` then chopped everything from `cma_run_provider()` to EOF — destroying previously-written provider aliases and any `claudeN` aliases that follow the function block. This silently corrupted the alias file on multi-provider / multi-account hosts. Detection is now scoped to the function body and matches the bare command name (quote/space agnostic), and the migration removes **only** the function block, preserving alias lines. This was the single root cause of the failures across `test_providers.sh`, `test_claude.sh`, and `test_add_remove.sh`.
- **set -u abort while sourcing the keys file** — provider sync sourced `~/api_keys.sh` inside a `set -euo pipefail` subshell. A dangling reference in the user's keys file (e.g. `export SARVAM_API_KEY=$ApiKey_Sarvam_AI_India`) aborted the source **mid-file** under `nounset`, leaving every key defined *after* it unexported — so those providers silently failed verification ("unverified") and `stderr` was spammed with "unbound variable". Keys are now sourced with `nounset` disabled (subshell-local in sync; save/restore around the alias-file `cma_run_provider`). Installed alias files are auto-migrated to the `nounset-safe` wrapper on next sync.
- **macOS / bash-3.2 portability of the test harness** — `tests/run-all.sh` used `mapfile` (bash 4+), so the **entire suite failed to run on stock macOS**. Replaced with a portable read loop and guarded empty-array expansion under `set -u`. Same fix applied to `test_lib.sh` and `tests/lib/sandbox.sh` (empty \$

{arr[@]} expansions are unbound on bash 3.2). The suite now runs green on macOS bash 3.2.

Added

- **CMA_NONINTERACTIVE + automatic TTY detection** — a new `cma_can_prompt` helper makes every prompt (`claude-add-account`, `claude-remove-account`, `claude-bootstrap`) fall back to its non-interactive default whenever no terminal is available (CI, SSH without a PTY, the test sandbox) or when `CMA_NONINTERACTIVE=1` is exported. Toolkit execution is now **always non-interactive off a terminal**. Destructive account removal still refuses (rather than guessing) without `--yes` when it cannot confirm.
- **Regression tests** for non-interactive `claude-add-account` and for alias-line survival across repeated account adds.
- **`test_export.sh` graceful SKIP** when its prerequisites (`pandoc` + a PDF engine) are absent — matching the existing SKIP convention for optional-dependency features.

Multi-host rollout (nezha · mistborn.local · thinker.local · amber.local)

- Distributed `~/api_keys.sh` to every host via a **no-loss merge** (host-local keys preserved — e.g. `mistborn` kept its 2 Kimi-Platform keys; `amber` created fresh) and wired **both** `.bashrc` and `.zshrc` to source it on every host.
- Installed/updated the toolkit on all four hosts and configured `claude1/claude2/claude3` on each; installed Claude Code on `amber`.
- Ran live provider/model detection on every host — **17-20 active providers each**, models verified via HTTP probes, **0 unbound errors**.

Verified

- `scripts/tests/run-all.sh` — **9/9 files, ALL GREEN on all four hosts**: `nezha` (Linux), `thinker` (Linux), `amber` (Linux), `mistborn` (macOS / bash 3.2).
- Cross-host: both rc files source `api_keys.sh`; `claude1/2/3` + `poe/deepseek/xiaomi` aliases present on every host.

v1.7.5 — 2026-06-28 — Cross-provider / resume session visibility fix

Fixed

- **Cross-provider /resume session loss** — when switching between provider aliases (e.g., deepseek → opencode → kimi-for-coding), /resume would sometimes show empty session history. Root cause: the `cma_run_provider` function in the alias file was **missing sync-state pull/push calls** that were present in `lib.sh`. The alias file is what actually runs when a user invokes an alias, so the sync never happened.
- **Migration for outdated alias files** — added automatic detection and regeneration of outdated `cma_run_provider` functions in `lib.sh`. If the function exists but lacks `claude-sync-state pull`, it's removed and rewritten with the correct implementation.
- **Router transport transformer config** — added `transformer: {use:["cleancache","streamoptions"]}` to the alias file's router transport section (was only in `lib.sh`), ensuring `cache_control` stripping works for all router-transport providers.

Root Cause Analysis

The `cma_run_provider` function in `lib.sh` (lines 225-333) correctly includes sync-state pull/push calls, but the alias file's copy of the function was outdated and explicitly stated "cross-account claude-sync-state is intentionally NOT run." This meant: 1. Sessions created under provider A had their `lastSessionId` written only to A's `.claude.json` 2. When switching to provider B, B's `.claude.json` still had its own (different) `lastSessionId` 3. /resume read B's `lastSessionId` and couldn't find A's session

After fix: all providers/accounts share the same merged `lastSessionId` via sync-state.

Verified

- **Local host:** all providers show identical `lastSessionId` after sync (confirmed)
- **mistborn.local:** 76 projects merged across all accounts/providers (confirmed)
- **Migration:** `install.sh` correctly detects and fixes outdated alias files on both hosts

Tests

- Cross-alias session visibility (Section 5): **ALL PASS**

- Existing test suite: session-related tests pass

v1.7.4 — 2026-06-26 — Kimi provider fix + AWS IaC MCP disabled by default

Fixed

- **Kimi Code provider base URL** in `scripts/providers/overrides.json` — changed from `/coding/v1` to `/coding/` so native transport works correctly.
- **AWS IaC MCP timeout** — removed `aws-dev-toolkit/awsiac` from the default OpenCode MCP allowlist in `scripts/claude-opencode-sync.sh`. The server consistently timed out on connection and is now configured but disabled by default.

Changed

- Regenerated `Claude_Multi_Account_Fine_Tuning.{html,pdf,docx}` from current markdown.
- Refreshed proof artifacts in `scripts/tests/proof/`.

Tests

- Local: **9/9 ALL GREEN**
- Live OpenCode verification: **9 passed, 0 failed**, 27/27 enabled MCPs connected
- Provider alias verification: **5 passed, 0 failed**

v1.6.6 — 2026-06-21 — TOON integration for token-efficient prompts

Added

- **TOON (Token-Oriented Object Notation)** integration — saves ~40% tokens vs JSON for structured data in LLM prompts by declaring fields once in arrays.
- `scripts/toon.mjs` — Node.js TOON utility (encode/decode/demo)
- `scripts/toon_encode.py` — Python wrapper for TOON encoding
- `docs/TOON_Integration.md` — comprehensive guide on using TOON with Claude Code
- `package.json` — `@toon-format/toon v2.3.0` dependency

Token Savings

- File listings: ~39% fewer tokens

- Tool definitions: ~40% fewer tokens
- User records: ~42% fewer tokens
- Accuracy: 76.4% (vs JSON's 75.0%)

Note

TOON formats message CONTENT for token savings. API transport remains JSON (providers require it). HTTP/3 and compression require provider-side support.

Tests

- 8/8 ALL GREEN

v1.6.5 — 2026-06-21 — Poe proxy fix (alias file + install)

Fixed

- **Poe proxy not starting from alias** — proxy logic was only in `lib.sh`, not in the alias file's `cma_run_provider` function. The alias file is what actually runs when a user invokes an alias. Added proxy detection + auto-start to the alias file.
- **install.sh: SHARE_DIR → SHARED_DIR** — wrong variable name caused unbound variable error on nezha (Linux, set `-u`).
- **install.sh: auto-copy proxy scripts** to `~/.local/share/.../proxy/` during install.

Verified

- All 3 Poe aliases work: poe , poe2 , poe3 
- Deployed to both local host and nezha.local

Tests

- 8/8 ALL GREEN

v1.6.4 — 2026-06-21 — Poe proxy fix for tool compatibility

Fixed

- **Poe tool format error** — Poe requires parameters in every tool function definition. Claude Code sometimes omits it (valid in

Anthropic format, invalid for Poe). Added `poe_proxy.py` that auto-fixes tools before forwarding to Poe API.

- **Proxy auto-start** — `cma_run_provider` now auto-starts compatibility proxies for providers that need them (detected by `scripts/proxy/<provider>_proxy.py`).

Verified

- All 3 Poe aliases work through proxy: `poe` , `poe2` , `poe3` 

Tests

- 8/8 ALL GREEN

v1.6.3 — 2026-06-21 — Poe provider (382 models, 3 aliases)

Added

- **Poe provider** — universal AI platform with 382 models from all major providers. OpenAI-compatible API at <https://api.poe.com/v1>. Chat, code, image gen, video gen, TTS, STT, and more.
- **3 aliases**: `poe` (claude-sonnet-4.6 + gpt-5.4-mini), `poe2` (gpt-5.5 + deepseek-v4-pro-e), `poe3` (grok-4 + gemini-3.1-pro)
- **key-aliases**: `POE_API_KEY` + `ApiKey_Poe` → `poe`
- **Tool calling verified** on claude-sonnet-4.6, gpt-5.4-mini, deepseek-v4-pro-e, grok-4
- **382 models categorized**: 130 chat/reasoning, 16 code, 40 image gen, 17 video gen, 12 TTS, 1 STT, 166 other
- **Documentation**: full Poe section in `Provider_Aliases_User_Guide.md`

Verified

- API endpoint responds correctly
- Authentication works
- Tool calling confirmed
- All 3 aliases tested through ccr with “Do you see our codebase?” — all YES

v1.6.2 — 2026-06-21 — Chutes provider documentation + model update

Changed

- **Chutes provider models updated** — catalog was stale. Chutes now offers 13 TEE (Trusted Execution Environment) models. Updated strong=zai-org/GLM-5.2-TEE, fast=Qwen/Qwen3.6-27B-TEE.
- **Chutes documentation** added to Provider_Aliases_User_Guide.md with full model table, TEE explanation, pay-per-use note, and setup instructions.

Verified

- Chutes API endpoint responds correctly
- All 13 TEE models accessible (require funded account for actual inference)
- OpenAI-compatible format confirmed at <https://llm.chutes.ai/v1>

v1.6.1 — 2026-06-21 — cache_control fix + E2E tests

Fixed

- **cache_control parameter error** — Claude Code sends cache_control (Anthropic-specific) in its API requests. ccr forwarded this to OpenAI-compatible endpoints which reject it with HTTP 422. Fixed by adding ccr's built-in cleancache transformer to every provider config, which strips cache_control before forwarding to the provider.

Added

- **alias_e2e_test.py** — end-to-end alias verification script that tests each alias by sending requests through ccr and verifying responses work without errors.

Verified working (all aliases tested with “Do you see our codebase?”)

- opencode (north-mini-code-free):  YES
- opencode2 (big-pickle):  YES
- opencode3 (nemotron-3-ultra-free):  YES
- deepseek (native transport):  YES

- deepseek2 (router transport):  YES
- xiaomi (native transport):  YES
- zai-coding-plan (router transport):  YES

v1.6.0 — 2026-06-21 — Multi-alias provider system

Added

- **Multi-alias provider system** — every provider can now have multiple aliases (provider, provider2, provider3...) exposing ALL working models, not just the top 2. Verified via live HTTP probes with anti-bluff detection.
- **model_verify.py** — comprehensive model verification & scoring engine. Tests every model for a provider via HTTP probes, scores on 7 dimensions (existence 25pts, tool_call 20pts, reasoning 15pts, context_window 15pts, streaming 10pts, latency 10pts, free_tier 5pts). Anti-bluff detection prevents false positives (HTTP 200 with error body, empty responses, boilerplate errors). 24h verification cache to avoid re-testing.
- **providers_generate.py** — multi-alias generation from verified models. Pairs models into alias groups of 2 (strong + fast), handles odd count (last model reused for both positions), single model (used for both positions). Generates env files, shell aliases, and overrides.json entries.
- **claude-providers.sh --multi** — new flag for sync that triggers the full verification + multi-alias generation pipeline. Additional flags: --max-aliases (default 5), --min-score (default 25), --verify-concurrency (default 5).
- **Endpoint normalization** — /anthropic endpoints auto-converted to /v1 for OpenAI-compatible probing during verification.
- **Submodules updated** to helix_translate-2.3.1: LLMsVerifier (ModelVerifier, Seed, xiaomi provider), challenges (anti-bluff §11.4, chaos/stress tests), containers (deploy-stack).

Changed

- Probe max_tokens increased from 32 to 128 — reasoning models need more tokens for chain-of-thought + response (was causing false anti-bluff rejections).
- User-Agent header added to HTTP probes (some APIs require it).

Full test suite

- 8/8 test files passed (ALL GREEN), 0 failures.

Usage

Standard sync (2 models per provider, as before)

```
claude-providers sync
```

Multi-alias sync (verify ALL models, create multiple aliases)

```
claude-providers sync --multi
```

With options

```
claude-providers sync --multi --max-aliases 10 --min-score 20
```

v1.5.1 — 2026-06-20 — Linux stat fix + nezha deployment

Fixed

- **stat -f %m on Linux** — the mtime cache check in `claude-providers.sh` used `stat -f %m || stat -c %Y` as an `||` chain. On Linux, `stat -f` succeeds (returning filesystem info, not mtime), so both outputs merged into garbage ("File: ...1781634386"), causing File: unbound variable under set -u. Fixed with case "\$ (uname -s)" to pick the correct flag per platform.

Deployment

- **nezha.local** (Linux x86_64) deployed and verified: 19 providers activated, 100/100 provider tests pass, 5/5 live verifier pass, cross-alias sync confirmed. Evidence in `scripts/tests/proof/90-nezha-deployment.txt`.

Full test suite

- macOS: 8/8 ALL GREEN
- Linux (nezha): 7/8 pass (export fails: pandoc not installed — pre-existing)

v1.5.0 — 2026-06-20 — Cross-alias session visibility

Added

- **Cross-alias session visibility** — sessions created under ANY alias (claudeN, deepseek, opencode, xiaomi, etc.) are now visible from every other alias via `/resume`. Memory, project settings,

and session data are fully shared across all accounts and providers.

- **claude-sync-state.sh extended** — now discovers provider dirs (`~/ .claude-prov-*`) alongside account dirs for its `.claude.json` merge. Provider sessions participate in the same lightweight jq merge that keeps account sessions in sync.
- **cma_run_provider sync-state hooks** — the provider wrapper now calls `claude-sync-state pull` before launch and `claude-sync-state push` after exit, matching the `cma_run` pattern. Previously provider sessions were intentionally excluded from sync; now they participate fully.
- **Sandbox test coverage:** 10 new assertions proving cross-alias merge (sessions from account→provider, provider→account, account→account all visible after sync). Providers test 90 → 100 assertions.
- **Live verification:** `lastSessionId` for a real project confirmed identical across all dirs (3 accounts + 1 provider). 61 projects merged in every `.claude.json`. Evidence in `scripts/tests/proof/80-cross-alias-sessions.txt`.

Changed

- `scripts/claude-sync-state.sh` — provider dirs included in merge targets
- `scripts/lib.sh` — `cma_run_provider` wrapper updated with `sync-state pull/push`
- Alias file `aliases.sh` — updated `cma_run_provider` function (re-installed)

Full test suite

- 8/8 test files passed (ALL GREEN), 0 failures. Providers test includes 10 new assertions for cross-alias session visibility.

How it works

1. `claude-sync-state pull` merges every account's + provider's `.claude.json` into the launching dir before Claude Code starts (including `lastSessionId`, `allowedTools`, MCP config, etc.).
2. Claude Code launches with the merged state — `/resume` sees all sessions.
3. `claude-sync-state push` merges the post-session `.claude.json` back out after exit, so the next alias to launch picks up the new session.
4. The `sessions/` directory was already shared via symlink — this release ensures `.claude.json` project settings are also merged.

Performance

- Adds ~1-2 seconds overhead per provider launch (jq merge of `.claude.json` across all dirs). Same overhead that `claudeN` aliases already have.

v1.4.0 — 2026-06-20 — OpenCode Zen provider alias

Added

- **opencode provider alias** — OpenCode Zen curated AI gateway with **21 free models** (all \$0 cost, all support tool calling + reasoning) and 49 paid models. The alias uses **router transport** (ccr) targeting the OpenAI-compatible endpoint `https://opencode.ai/zen/v1/chat/completions`.
- **Model overrides**: `strong` = `big-pickle` (free stealth model, 200K context, reasoning + `tool_call`), `fast` = `deepseek-v4-flash-free` (free, 200K context, reasoning + `tool_call`). Pinning is deliberate — auto-selection would pick `nemotron-3-ultra-free` (1M ctx) as `strong` and `trinity-large-preview-free` (131K, no reasoning) as `fast`, both suboptimal for coding workloads.
- **key-aliases.json mappings**: `ZEN_API_KEY` → `opencode` and `ApiKey_Opencode_Zen` → `opencode` (both key vars present in the user's keys file).
- **overrides.json pin**: `strong_model=big-pickle`, `fast_model=deepseek-v4-flash-free` (no `transport/base_url` override needed — catalog values are correct).
- **Sandbox test coverage**: resolver tests (key-alias mapping for both key vars, router transport from `@ai-sdk/openai-compatible` npm, `zen/v1` `base_url` from catalog, model override beats auto-selection, stale-model-never-selected guards) + sync e2e tests (env file, alias, config-dir + plugins symlink, account-detection exclusion, idempotency, no-secret-leak). Providers test 69 → 90 assertions.
- **Live endpoint verification**: GET `/v1/models` HTTP 200; POST `/v1/chat/completions` round trip HTTP 200 with correct text for `big-pickle` (stealth, cost=\$0, reasoning_content present) and `deepseek-v4-flash-free` (cost=\$0); additional free models (`mimo-v2.5-free`, `nemotron-3-ultra-free`, `north-mini-code-free`) all HTTP 200 with cost=\$0. Evidence in `scripts/tests/proof/70-zen-live.txt` (secret-free).
- **Docs**: dedicated `opencode` section in `docs/ProviderAliasesUserGuide.md` (full free models table, setup, usage, live-verified notes, stealth model explanation).

Changed

- `scripts/providers/key-aliases.json` and `scripts/providers/overrides.json` extended with the `opcode` entries (config-only; no code changes — same dynamic pattern as Xiaomi v1.3.0 / Z.AI v1.2.0 / DeepSeek).

Full test suite

- 8/8 test files passed (ALL GREEN), 0 failures. Providers test includes 21 new assertions for `opcode`.

Honest notes

- The alias uses router transport (`ccr`) because Zen's free models use OpenAI-compatible format (`/v1/chat/completions`), not Anthropic native format. This adds a `ccr` dependency that native-transport aliases (`deepseek`, `xiaomi`) don't have.
- Big Pickle is a stealth model — the actual model served may vary (observed as `deepseek-v4-flash`). This is by design per OpenCode's documentation.
- The same pre-existing `~/api_keys.sh set -u` issue affects the in-process verifier for all providers; authoritative proof is the direct HTTP round trip.
- The 2 pre-existing, environmental `opcode-skill-discovery` failures in `run-proof.sh` remain unchanged (unrelated to this work).

v1.3.0 — 2026-06-19 — Xiaomi MiMo provider alias

Added

- **xiaomi provider alias** — Xiaomi MiMo via the **Anthropic-native endpoint** `https://api.xiaomimimo.com/anthropic` (POST / `anthropic/v1/messages`). Unlike most providers in this toolkit, MiMo exposes a genuine native Anthropic endpoint that accepts `Authorization: Bearer`, so the alias uses **native transport** with no `claude-code-router (ccr)` dependency — the same direct-launch model as `deepseek`.
- **Model overrides**: `strong` = `mimo-v2.5-pro` (flagship, 1M context, reasoning, tool-call), `fast` = `mimo-v2-flash` (256K, cheapest tier). Pinning is deliberate — `models.dev` lists a `mimo-v2.5-pro-ultraspeed` id the **live API does not serve**, so the override guarantees only live-served ids are used.
- **key-aliases.json mapping**: `XIAOMI_MIMO_API_KEY` → `xiaomi` (the user's key-var name does not match the `models.dev` provider's documented `XIAOMI_API_KEY` env).

- **overrides.json pin:** native transport, /anthropic base_url, mimo-v2.5-pro / mimo-v2-flash.
- **Sandbox test coverage:** resolver tests (key-alias mapping, override forces native transport, /anthropic base_url beats catalog /v1, model pinning beats the stale ultraspeed entry, stale-id-never-selected guard) + sync e2e tests (env file, alias, config-dir + plugins symlink, account-detection exclusion, idempotency, no-secret-leak). Providers test 60 → 69 assertions.
- **Live endpoint verification:** GET /v1/models HTTP 200 (10 models); native /anthropic/v1/messages round trip HTTP 200 with correct text for both mimo-v2.5-pro and mimo-v2-flash; tool calling proven (finish_reason: tool_calls
◦ reasoning_content); streaming confirmed. Evidence in scripts/tests/proof/60-xiaomi-live.txt (secret-free).
- **Docs:** dedicated xiaomi section in docs/ Provider_Aliases_User_Guide.md (model table, setup, usage, live-verified notes).

Changed

- scripts/providers/key-aliases.json and scripts/providers/overrides.json extended with the xiaomi entries (config-only; no code changes — same dynamic pattern as Z.AI v1.2.0 / DeepSeek).

Full test suite

- 8/8 test files passed (ALL GREEN), 0 failures. Providers test includes 9 new assertions for xiaomi. Live provider verifier 5/5 PASS.

Honest notes

- The only failures in the repo's run-proof.sh are 2 **pre-existing, environmental** opencode-skill-discovery checks, unrelated to Xiaomi (zero opencode files changed by this release; they fail identically when run standalone).
- The in-process LLMsVerifier step reports (unverified) for every provider because ~/api_keys.sh has a pre-existing unrelated unbound variable on a different provider's key under set -u; authoritative proof is the direct native-endpoint round trip (HTTP 200), recorded in the evidence file.

v1.2.0 — 2026-06-19 — Z.AI Coding Plan provider alias

Added

- **zai-coding-plan provider alias** — OpenAI-compatible router transport via `https://api.z.ai/api/coding/paas/v4` (Coding Max-Yearly Plan endpoint).
- **Model overrides**: strong = glm-5.2 (flagship 1M context reasoning model, free on plan), fast = glm-4.7 (204k context, tool_call, 0 cost).
- **key-aliases.json mapping**: `ZAI_API_KEY` → `zai-coding-plan` (targets the coding plan API endpoint instead of the general `z.ai paas` endpoint).
- **overrides.json pin**: overrides auto-selected strong/fast models for the coding plan.
- **Sandbox test coverage**: resolver tests (env-key matching, coding endpoint, router transport, glm-5.2/glm-4.7 model selection) + sync e2e tests (env file, alias, model overrides).
- **Live endpoint verification**: HTTP 200 at `/models` (8 models discovered), curl test of glm-4.7 chat completion confirmed operational.
- **ccr integration**: provider auto-registered in `~/.claude-code-router/config.json` as the active default route.

Changed

- `overrides.json` extended with `zai-coding-plan` section for model pinning.

Full test suite

- 8/8 test files passed (ALL GREEN), 0 failures. Provider tests include 5 new assertions for `zai-coding-plan`.

v1.1.0 — 2026-06-16 — Distributed infrastructure + provider verification

Headline: stand up the full LLMsVerifier System on a remote host for heavy testing against **real production LLM services**, plus end-to-end provider aliases proven on two hosts and two transports.

Added

- **containers + challenges submodules** (submodules/) — the distributed-boot orchestrator and its sibling. `helix-deps.yaml` confirms containers has zero own-org submodule deps.
- **Remote host registration** — `config/containers/nezha.env` registers `nezha.local` as a remote boot/test host (SSH key, podman runtime).
- **LLMsVerifier deployment overlays** (`config/containers/llmsverifier/`):
 - `docker-compose.app.yml` — the `llm-verifier` API (cgo image, config mount, `/api/health` healthcheck, loopback, fail-fast secrets).
 - `docker-compose.infra.yml` — observability tier: prometheus + grafana (auto-provisioned datasource + dashboard) + node-exporter. **No DBs** (the app uses SQLite; postgres/redis were unused and removed).
 - `Dockerfile.nezha / Dockerfile.mv` — cgo nested-module builds for the server + the model-verification tool.
 - `patches/0001..0005` — upstream LLMsVerifier fixes (see PR #2 below).
- **Deployment guide** `config/containers/llmsverifier/README.md` and the **Provider Aliases User Guide** `docs/Provider_Aliases_User_Guide.md` (HTML/PDF/DOCX exports included).
- **QA evidence** `docs/qa/20260616-infra/` — verification proofs, endpoint coverage, security posture, observability, per-provider sweeps, dual-host end-to-end alias proofs.

Changed

- **Provider session accent color: orange → purple** across spec, guide, and the long-form doc. (Claude Code 2.1.178 cannot persist a default `/color`, so this is the documented default + a manual `/color purple` — a platform limit.)
- `claude-add-account` consolidated onto the shared `cma_link_shared_items` helper (single `CMA_SHARED_ITEMS` source).
- `claude-export-docs` now also emits **DOCX** (HTML/PDF/DOCX).

Fixed (LLMsVerifier — shipped as PR #2, applied to deployed builds)

- **Auth header missing** — verification requests sent no Authorization header → HTTP 401 for every provider. Now Bearer <key>.
- **cohere 405** — switched to the OpenAI-compatible endpoint (`api.cohere.ai/compatibility/v1`). Verifies at score 1.00.
- **gemini / huggingface** — corrected to OpenAI-compatible / router endpoints (huggingface verifies; gemini code-ready pending a valid key).

- **model-id strictness** — verifies a requested id directly when not in the discovered list (no premature `model_not_found`).
- **no /metrics** — added `GET /api/metrics + /metrics` (stdlib Prometheus).
- **provider-session sync-state noise** — `cma_run_provider` no longer runs cross-account sync-state on isolated provider dirs.

Verified live (real “Do you see my code?” against production APIs)

- **9 providers verified:** DeepSeek, Groq, Mistral, Cerebras, Novita, NVIDIA, Cohere, Codestral, HuggingFace.
- **Both transports, both hosts:** native (DeepSeek) + router (Novita via ccr) on macOS and on nezha.
- Account-side failures (402/401/429/403) and non-OpenAI providers documented honestly; excluded under “valid users only” but kept fully supported.

Safety

- Provider dirs (`~/.claude-prov-*`) excluded from account detection — existing claudeN accounts and `claude-add-account` untouched.
- Secrets only in the keys file + on-host `mode-600 .env`; never in the repo. All published ports bound to loopback.

v1.0.0 — 2026-06-16 — Dynamic provider-alias generator

First tagged release. `claude-providers` creates per-provider Claude Code aliases (DeepSeek, Groq, GLM, ...) from your keys file pointed at each provider’s strongest model — fully dynamic via `models.dev` + the `LLMsVerifier` submodule, hybrid native/claude-code-router transport, full lifecycle + tests + docs. See `docs/Provider_Aliases_User_Guide.md`.

v1.6.7 — 2026-06-21 — Poe proxy fix for all aliases

Fixed

- **Poe proxy not starting for poe2/poe3** — proxy detection used exact provider ID (`poe2_proxy.py`) which doesn’t exist. Fixed to check base name too (`poe_proxy.py` for poe2, poe3 aliases).
- **lib.sh:** base proxy detection with `${CMA_PROVIDER_ID%[0-9]*}`

- **alias file:** same fix applied

Verified

- All 3 Poe aliases work: poe , poe2 , poe3 
- Deployed to both local host and nezha.local

Tests

- Local: 8/8 ALL GREEN
- nezha.local: 7/8 (pandoc missing — pre-existing)

v1.6.8 — 2026-06-21 — Poe proxy gzip fix

Fixed

- **Poe proxy gzip decompression** — Poe API returns gzip-compressed responses but the proxy tried to read them as UTF-8 without decompressing, causing UnicodeDecodeError. Added gzip decompression for both success and error responses.

Verified

- poe (claude-sonnet-4.6):  YES
- poe2 (gpt-5.5):  YES
- poe3 (grok-4):  Different error (Grok-4 schema validation, not tools format)

Tests

- Local: 8/8 ALL GREEN
- nezha.local: 8/8 ALL GREEN

v1.6.9 — 2026-06-21 — Poe proxy \$ref fix for Grok-4

Fixed

- ****Poe proxy \$ref resolution**** — Claude Code sends tool schemas with ``$ref`references to`$defs``. Grok-4 and some providers don't support ``$ref`in tool schemas`.

Added `resolve_refs()` function that extracts `$defs`, resolves all `$ref` references to inline definitions, and removes `$defs`.

Verified

- poe (claude-sonnet-4.6):  YES
- poe2 (gpt-5.5):  YES
- poe3 (grok-4):  YES (was failing, now works)

Tests

- Local: 8/8 ALL GREEN
- nezha.local: 8/8 ALL GREEN

v1.7.0 — 2026-06-22 — Poe proxy complete fix (all aliases verified)

Fixed

- **Poe proxy shared directory** — the proxy at `~/.local/share/.../proxy/poe_proxy.py` was the OLD version without gzip and `$ref` fixes. `install.sh` copies from `scripts/` but the shared dir still had the old version. Fixed by ensuring updated proxy is copied to shared directory.
- **install.sh** now copies proxy scripts during installation (already in place)

Verified (all three aliases through full Claude Code flow)

- poe (claude-sonnet-4.6):  YES
- poe2 (gpt-5.5):  YES
- poe3 (grok-4):  YES

Root Cause Analysis

The proxy had three issues: 1. **gzip** — Poe returns gzip-compressed responses, proxy didn't decompress 2. **\$ref** — Claude Code sends tool schemas with `$ref`, Grok-4 doesn't support them 3. **shared dir** — Updated proxy wasn't copied to shared directory

All three fixed and verified.

Tests

- Local: 8/8 ALL GREEN

- nezha.local: 8/8 ALL GREEN

v1.7.1 — 2026-06-22 — Full validation + release

Fixed

- **Port-ready check** for proxy startup — replaced `sleep 1` with polling loop (`lsof -i`) ensuring proxy is listening before ccr config is written
- **Claude alias regression test** — 11 assertions proving claudeN aliases use `cma_run` (no proxy/transformer code), providers use `cma_run_provider`
- **Command injection fix** in `verify_aliases_live.sh` — replaced `bash -c` subshell with safe indirect expansion

Tests

- Local: **9/9 ALL GREEN** (new: `test_claude.sh` — 11 assertions)
- nezha.local: 8/9 (export fails — pandoc missing)

Release

- v1.7.1 — pushed to github, gitlab, gitflic, gitverse

v1.7.2 — 2026-06-22 — Claude alias verification, full release

Added

- **Claude alias verification** in `verify_aliases_live.sh` — tests claude1/2/3 alongside provider aliases
- **TOON tested** on all aliases — verified working

Tests

- Local: **9/9 ALL GREEN**
- nezha.local: 8/9 (pandoc missing)
- All claude1/2/3:  OK
- All provider aliases: verified